

Features of Java

- Simple–Easy to understand
- Open Source–Free
- Object Oriented
- Platform Independent
- Portable–Ability to run on different Systems
- Compiled and then Interpreted
- Distributed and Secure

Types of java

JAVA2SE–Standard Edition–This is provide basics and for desktop application .

JAVA2EE–Enterprise Edition–This is for web application development.

JAVA2ME–Micro Edition–This is develop the application for mobile, tablet..

Working environment of Java

- Download **jdk1.8** and install.
- Download **netbeans** for **java & j2ee** developer and install.
- Download **mysql** and install.
- Download **mysql-connector jar** for java
- The connector “jar” place your library folder on netbeans.

Java Programs

Java programs fall into two categories:

1. Applications
2. Applets

Applications

An application in java is a program that you can execute from any operating system prompt.

Stand-alone applications can either be,

1. Window based applications
2. Console based applications

Applets

Applets are programs that execute inside a web page.

How to Run Applet

- runs in
 - appletviewer (test utility for applets)
 - The Applet viewer takes care of providing the environment and calling several of the applet's methods.
 - Web browser (IE, Communicator)
 - Modern browsers come with applet capabilities.



Java Applets

JAVA PROGRAMMING INTRODUCTION

Object Oriented Programming (OOP) Language. Java was developed by **Sun Microsystems** and later undertaken by Oracle Corporation.

Java is freely downloadable whereas .NET technology is not available for free.

Once the Java program is written, we can Run/ Reuse anywhere.

It is platform independent.

Java supports all design patterns which are predefined.

Object Oriented Program's Concepts

- Object Oriented Programming Concepts

Class

Objects

Data Encapsulation

Data Abstraction

Inheritance

Polymorphism

Dynamic Binding (Map between super and sub class)

Class

- It provides way to bind the data and its associated method

Syntax: class classname

Example Class Hierarchy

```
public class Dog
{
    String breed;
    int ageC;
    String color;
    void barking()
    {
    }
    void hungry(){
    }
    void sleeping(){
    } }

// Class Declaration
// Class Definition wo main()
// Variable Declaration

// Method Declaration
// Method Definition
```

Creating Objects

- The new keyword. Object Instantiation.

Syntax:

Classname objectname=new Classname();

Example:

```
public class Test{
    public Test(String name){
        // This constructor has one parameter, name.
        System.out.println("Passed Name is :" + name );
    }
    public static void main(String []args){
        // Following statement would create an object Test
        Test obj = new Test( "Aadhithyan" );
    }
}
```

Data Encapsulation

- Information Hiding

Data Abstraction

- Ignore the irrelevant details and concern on the essentials

Inheritance

- Single Inheritance
- Multilevel Inheritance
- Hierarchical Inheritance
- Multiple Inheritance (Java does not support)
- Hybrid Inheritance (Java does not support)

Polymorphism (One form in many form)

- Compile Time Polymorphism
- Run Time Polymorphism (Supported by Java)

Java Fundamentals

- Javac Compiler (javac filename.java)
- Java Interpreter (java filename)
- Appletviewer (appletviewer url of .html)
- Jdb Tool (jdb classname)
- Javap Disassembler (javap path)
- Javadoc Tool (javadoc list of files)
- Javah Tool (javah class filename)

Creating Datatypes

1. Primitive/Standard Datatypes

2. Abstract/Derived Datatypes

Data type	Size	Range	Description
Byte	8 bit	-128 to 127 » 0 to 255 » Unsigned	Signed Numeric data types that can hold only whole numbers
Short	16 bit	-32768 to +32767	
Int	32 bit	-2^{31} to $2^{31}-1$	
Long	64 bit	-2^{63} to $2^{63}-1$	
Float	32 bit	+ /- about 10^{39}	Numeric data types that can hold decimal values also
Double	64 bit	+/-about 10^{317}	
Char character	16 bit		Can hold a single
Boolean	8 bit		True or False

Rules for naming variables in Java

- Must not be a keyword in java
(A keyword is a reserved word used by Java.
Keywords have a special meaning in the Java language
and must not be used as class, variable, or method names).
- Must not begin with a digit
- Must not contain embedded spaces
- Can contain characters from various language alphabets

Variable naming Conventions in Java

- Variable names should be meaningful. The names should reflect the data the variables contain.
Example: To store the age of an employee, the variable can be empAge.
- Variable names are nouns and begin with a lowercase letter.
- If a variable name contains two or more words, join the words and begin each word with an uppercase letter. The first word starts with a lowercase letter.

Access Modifiers

Public	—	Access within its own and other package
Private	—	Access within a method
Protected	—	Access within a package
Default	—	with in a package

Constructor

Rules for Constructor

- A Constructor always has the same name as that of the class name.
- Constructor does not have return type.

Types of Operator

- Arithmetic Operators

Operator	Meaning	Example
+	Addition	8+10
-	Subtraction	10-8
*	Multiplication	20*84
/	Division	10/5
%	Modulus	10%6

- Assignment Operators

Expression	Meaning
$x+=y$	$x=x+y$
$x-=y$	$x=x-y$
$x*=y$	$x=x*y$
$x/=y$	$x=x/y$
$x=y$	x equals to y

- **Increment and Decrement Operators**

++a (Pre Increment)

a++ (Post Increment)

--a (Pre Decrement)

a- (Post Decrement)

- **Logical Operators**

AND (& or &&)

OR (| or ||)

XOR (^)

NOT (!)

Bitwise Operators

Operator	Meaning
&	Bitwise AND
	Bitwise OR
^	Bitwise XOR
<<	Left Shift
>>	Right Shift
>>>	Zero fill Right Shift
~	Bitwise Complement
<<==	Left Shift assignment
>>==	Right Shift assignment
>>>=	Zero fill Right shift assignment
x&=y	AND assignment
a =y	OR assignment
X^=y	XOR assignment

- **Conditional Operator**

Syntax: `value=(UserSetIt ? UsersValue : defaultValue);`

Precedence of Operators

The order of Operators and its precedence is given below.

()
!, unary-, unary+ , ++, -- (post)
++, -- (pre)
* / %
+ -
<<= > =>
== !=
&&
||
=

Highest Precedence



Lowest Precedence

Example: `a / b * c` – means

`{a / b} * c` –not `a / (b * c)`

Java Statements

Statements

The two basic statements are:

- Expression statements
- Declaration statements
- Expression statements terminate with semicolon.
- Declaration statements appear anywhere inside the block.

Comments

Comments within source code exist for the convenience of human programmers.

There are three kinds of comments:

`// comment`

`/* comment */`

`/** comment */ for document...`

Keywords

Keywords have special meaning within the programming language.
Here is the list of keywords:

Abstract	default	if
Boolean	do	implements
Break	double	imports
Byte	else	int
Char	extends	interfaces
Class	final	long

Arrays

- Arrays are used to store number of contiguous values.

Declaring an Array

Syntax: `data_type[ ] variablename;`

Example:

```
int[ ] numbers;
```

```
//Declaration
```

```
numbers = new int [10];
```

```
//Memory allocation
```

- Arrays are used to store number of contiguous values.

Types of Array:

Single dimensional array: `int[]`

Two dimensional array: `int [][]`

Multidimensional array: `int [][][]`

Example of Two dimensional Array:

```
class TestArray{
public static void main(String...a){
    //int a[3][3]={{1,2,5},{7,8,9},{70,80,90}};

    int[][] as = new int[][] {{1 1, 1 2},{1,2}};

    for(int i=0;i<2;i++){
        for (int j=0;j<2;j++){
            System.out.println(as[i][j]);
        }
    }
}}
```

Decision Constructs

- if else Construct

SYNTAX

```
if(expr)
{
    Statements;
}
else
{
    Statements;
}
```

Example of If-else

```
if(iNum>iNum2)
{
iMax=iNum1;
}
else
{
iMax=iNum2;
}
```

- Iterative Constructs (Looping Statements)

for loop Syntax:

```
for(initialization_expr;test_epr;increment/decrement_expr)
{
    Statements;
}
```

- Iterative Constructs (Looping Statements)

Enhanced for loop Syntax:

```
for(variable: arrays/collections)
{
    Statements;
}
```

Example:

```
class CreatePattern    {
    public static void main(String args[])    {
        int a[]={1,2,3,4,5,6};
        for(int j:a){
            System.out.println(j);
        }
    }
}
```

```
import java.io.*;
class CreatePattern
{
    public static void main(String args[])
    {
        for(int i=0;i<5;i++)
        {
            for(int j=0;j<5;j++)
            {
                System.out.print("* ");
            }
            System.out.println(" ");
        }
    }
}
```

Command Prompt

```
F:\Hari\JAVA\ExamplesII>javac CreatePattern.java
```

```
F:\Hari\JAVA\ExamplesII>java CreatePattern
```

```
* * * * *  
* * * * *  
* * * * *  
* * * * *  
* * * * *
```

```
F:\Hari\JAVA\ExamplesII>_
```

Create a java program to print the Decrease Triangle

```
import java.io.*;
class DecreaseTriangle
{
    public static void main(String args[])
    {
        for(int i=0;i<5;i++)
        {
            for(int j=i;j<5;j++)
            {
                System.out.print("* ");
            }
            System.out.println(" ");
        }
    }
}
```



Command Prompt

```
: \Hari\JAVA\ExamplesII> javac DecreaseTriangle.java
```

```
: \Hari\JAVA\ExamplesII> java DecreaseTriangle
```

```
* * * *
```

```
* * *
```

```
* *
```

```
*
```

```
: \Hari\JAVA\ExamplesII>
```

```
: \Hari\JAVA\ExamplesII> _
```

Create a java program to print Increase Triangle

```
import java.io.*;
class IncreaseTriangle
{
    public static void main(String args[])
    {
        for(int i=0;i<5;i++)
        {
            for(int j=0;j<=i;j++)
            {
                System.out.print("* ");
            }
            System.out.println(" ");
        }
    }
}
```

Command Prompt

```
F:\Hari\JAVA\ExamplesII>javac IncreaseTriangle.java
```

```
F:\Hari\JAVA\ExamplesII>java IncreaseTriangle
```

```
*
```

```
* *
```

```
* * *
```

```
* * * *
```

```
* * * * *
```

```
F:\Hari\JAVA\ExamplesII>
```

Switch

Switch statement allows you to transfer control to a labeled entry point in a block of statements

Syntax:

```
switch(expression)
{
    case n: statements
    case m: statements
    .
    .
    default: statements
}
```

```
public class SwitchDemo
{
    public static void main(String[] args)
    {
        int month=13;
        String monthString;
        switch(month)
        {
            case 1: monthString="January";
                    break;
            case 2: monthString="February";
                    break;
            case 3: monthString="March";
                    break;
            case 4: monthString="April";
                    break;
            case 5: monthString="May";
                    break;
        }
    }
}
```

```
case 6: monthString="June";  
    break;  
case 7: monthString="July";  
    break;  
case 8: monthString="August";  
    break;  
case 9: monthString="September";  
    break;  
case 10: monthString="October";  
    break;  
case 11: monthString="November";  
    break;  
case 12: monthString="December";  
    break;  
default: monthString="Invalid Month";  
    break;  
} System.out.println(monthString); }}
```



Command Prompt

```
F:\Hari\JAVA\ExamplesII>javac SwitchDemo.java
```

```
F:\Hari\JAVA\ExamplesII>java SwitchDemo
```

```
August
```

```
F:\Hari\JAVA\ExamplesII>java SwitchDemo
```

```
August
```

```
F:\Hari\JAVA\ExamplesII>javac SwitchDemo.java
```

```
F:\Hari\JAVA\ExamplesII>java SwitchDemo
```

```
Invalid Month
```

while

SYNTAX

```
while(Boolean-expression)
{
    //statements
}
```

do-while

```
do
{
    //statement
}
while(Boolean-expression);
```

do-while

```
public class Test
{
    public static void main(String[] args)
    {
        int i=0;
        do
        {
            System.out.println("Input Value is:"+i);
            i++;
        }
        while(i<5);
    }
}
```



Command Prompt

```
F:\Hari\JAVA\ExamplesII>javac Test.java
```

```
F:\Hari\JAVA\ExamplesII>java Test
```

```
Input Value is:0
```

```
Input Value is:1
```

```
Input Value is:2
```

```
Input Value is:3
```

```
Input Value is:4
```

```
F:\Hari\JAVA\ExamplesII>
```

```
F:\Hari\JAVA\ExamplesII>_
```

Labels

- Statements can be labeled to give them a name by which they can be referred.
- A label precedes the statement it names:
`Label: statement`
- Labels can be referred to only by the break and continue statements.

Break

- A break statement can be used to exit from any block, not just from a switch.

There are two forms of break:

The unlabeled break

`break;`

The labeled break

`break label;`

Continue

- A continue statement can be used only within a loop (for, while, do)
- It transfers control to the end of the loop's body.
- Like the break statement, the continue statement comes in an unlabeled form:
 continue;
and labeled form:
 continue label;

Return

- A return statement terminates execution of a method and returns to the invoker.
- If the method returns no value, a sample return statement will do:
 return;

- Write a program to perform the arithmetic process.
- Write a program to perform the addition and multiplication of two matrix.
- Write a program to find the greatest number given three number.